Methods, Collections & Exceptions:

Methods, Collections & Exceptions of java,

In Java, methods define reusable blocks of code, collections provide powerful data structures for managing groups of objects, and exceptions handle runtime errors gracefully. Together, they form the backbone of robust and maintainable Java applications.

1.Java Methods:

- **Definition**: A method is a block of code that performs a specific task and can be reused.
- **Syntax**:

```
returnType methodName(parameters) {
   // method body
   return value;
}
```

- **Types**:
    - **Instance methods**: Operate on objects.
    - **Static methods**: Belong to the class, not objects.
    - **Abstract methods**: Declared without implementation (used in abstract classes/interfaces).
    - **Final methods**: Cannot be overridden.

Example:

```
public int add(int a, int b) {
   return a + b;
```

}

2. Java Collections Framework:

The **Collections Framework** provides interfaces and classes to store and manipulate groups of objects efficiently.

Core Interfaces

Interface	Characteristics	Common Implementations
List	Ordered, allows duplicates	ArrayList, LinkedList, Vector
Set	No duplicates, unordered	HashSet, LinkedHashSet, TreeSet
Queue	FIFO order	PriorityQueue, LinkedList
Map	Key-value pairs, keys unique	HashMap, TreeMap, LinkedHashMap

Examples:

```
// List
List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");

// Set
Set<Integer> ids = new HashSet<>();
ids.add(101);
ids.add(101); // ignored
```

```
// Map
Map<Integer, String> studentMap = new HashMap<>();
studentMap.put(1, "Ravi");
studentMap.put(2, "Priya");
```

3.Java Exceptions:

Exceptions handle **runtime errors** and prevent program crashes.

Types of Exceptions:

- **Checked Exceptions**: Must be handled at compile time (e.g., IOException, SQLException).
- **Unchecked Exceptions**: Occur at runtime (e.g., NullPointerException, ArithmeticException).
- **Errors**: Serious issues not meant to be handled (e.g., OutOfMemoryError).

Handling Exceptions:

- **try-catch block**:

```
try {
   int result = 10 / 0;
} catch (ArithmeticException e) {
   System.out.println("Error: Division by zero!");
}
```

- **finally block**: Executes regardless of exception.
- **throw**: Used to explicitly throw an exception.
- **throws**: Declares exceptions a method might throw.

Custom Exception Example:

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

⚠️ **Key Considerations:**

- **Methods**: Promote reusability and modularity.
- **Collections**: Choose the right structure (ArrayList vs LinkedList, HashSet vs TreeSet) based on performance needs.
- **Exceptions**: Always handle gracefully to avoid program termination; use custom exceptions for domain-specific errors.

Learn Java methods, parameter passing, method overloading, and scope rules:

1. Java Methods:

- **Definition**: A method is a block of code that performs a specific task and can be reused.
- **Example**:

```
public int add(int a, int b) {
    return a + b;
}
```

2.Parameter Passing:

- **Definition**: Java always uses **pass-by-value**. This means a copy of the variable is passed to the method.
    - For **primitive types** (int, double, etc.), the actual value is copied.
    - For **objects**, the reference (memory address) is copied, so changes affect the original object.
- **Example**:

```
void changeValue(int x) {
   x = 50;
}
public static void main(String[] args) {
   int num = 10;
   changeValue(num);
   System.out.println(num); // Output: 10 (unchanged)
}
```

3. Method Overloading:

- **Definition**: Having multiple methods with the same name but different parameter lists (different number or type of parameters).
- **Example**:

```
class MathUtils {
   int add(int a, int b) {
      return a + b;
```

```
    }
    double add(double a, double b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

👉 The compiler decides which method to call based on arguments.

4.Scope Rules:

- **Definition**: Scope determines where a variable can be accessed.
- **Types**:
    - **Local variables**: Declared inside a method; accessible only within that method.
    - **Instance variables**: Declared inside a class but outside methods; tied to objects.
    - **Class variables (static)**: Declared with static; shared across all objects of the class.
- **Example**:

```
class Demo {
    int instanceVar = 10;       // Instance variable
    static int staticVar = 20;  // Class variable
```

```
    void show() {
        int localVar = 30;      // Local variable
        System.out.println(localVar);
        System.out.println(instanceVar);
        System.out.println(staticVar);
    }
}
```

✅ **Quick Recap**

- **Methods**: Reusable blocks of code.
- **Parameter Passing**: Always pass-by-value.
- **Method Overloading**: Same name, different parameters.
- **Scope Rules**: Define where variables can be accessed.

Master Java collection framework utilities including ArrayList, HashMap, and HashSet:

1.ArrayList:

- **Definition**: A resizable array implementation of the List interface. Unlike arrays, it can grow or shrink dynamically.
- **Key Features**:
  - Maintains insertion order.
  - Allows duplicate elements.
  - Provides fast random access.
- **Example**:

```
import java.util.ArrayList;
```

```
public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();
        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");

        System.out.println(names); // [Alice, Bob, Charlie]
    }
}
```

2.HashMap:

- **Definition**: A map implementation that stores data in **key-value pairs**. Keys must be unique, values can be duplicated.
- **Key Features**:
  - No guaranteed order of elements.
  - Fast lookups using hashing.
  - Allows one null key and multiple null values.
- **Example**:

```
import java.util.HashMap;

public class HashMapDemo {
    public static void main(String[] args) {
        HashMap<Integer, String> students = new HashMap<>();
        students.put(1, "Ravi");
```

```
        students.put(2, "Priya");
        students.put(3, "Arjun");


        System.out.println(students); // {1=Ravi, 2=Priya, 3=Arjun}
        System.out.println(students.get(2)); // Priya
    }
}
```

3.HashSet:

- **Definition**: A set implementation that stores unique elements using hashing.
- **Key Features**:
  - No duplicates allowed.
  - No guaranteed order of elements.
  - Allows one null element.
- **Example**:

```
import java.util.HashSet;

public class HashSetDemo {
    public static void main(String[] args) {
        HashSet<String> cities = new HashSet<>();
        cities.add("Delhi");
        cities.add("Mumbai");
        cities.add("Delhi"); // duplicate ignored
```

```
        System.out.println(cities); // [Delhi, Mumbai] (order may vary)
    }
}
```

☑ **Quick Recap:**

- **ArrayList** → Dynamic array, ordered, allows duplicates.
- **HashMap** → Key-value pairs, keys unique, fast lookup.
- **HashSet** → Unique elements, unordered, no duplicates.

Learn exception handling (try-catch, throw, throws) and basic file reading/writing:

1.Exception Handling:

Exceptions are unexpected events (like dividing by zero or missing files) that disrupt program flow. Java provides mechanisms to handle them gracefully.

Key Concepts

- **try-catch**: Wrap risky code in try, handle errors in catch.
- **throw**: Used to explicitly throw an exception.
- **throws**: Declares exceptions a method might throw.

Example:

```
public class ExceptionDemo {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // risky code
        } catch (ArithmeticException e) {
```

```
            System.out.println("Error: Division by zero!");
        }
    }


     // Example of throws
    public void readFile(String fileName) throws IOException {
        FileReader fr = new FileReader(fileName);
    }


    // Example of throw
    public void checkAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Age must be 18 or
above");
        }
    }
}
```

1.File Reading: Java provides classes in java.io and java.nio packages to read files**.**

Example (BufferedReader):

```
import java.io.*;

public class FileReadDemo {

    public static void main(String[] args) {

        try (BufferedReader br = new BufferedReader(new
FileReader("example.txt"))) {

            String line;

            while ((line = br.readLine()) != null) {

                System.out.println(line);

            }

        } catch (IOException e) {

            System.out.println("File not found or error reading file.");

        }

    }

}
```

3.File Writing:

Example (BufferedWriter):

```
import java.io.*;

public class FileWriteDemo {

    public static void main(String[] args) {

        try (BufferedWriter bw = new BufferedWriter(new
FileWriter("example.txt"))) {
```

```
            bw.write("Hello, Java File Handling!");
            bw.newLine();
            bw.write("This is the second line.");
        } catch (IOException e) {
            System.out.println("Error writing to file.");
        }
    }
}
```

☑ Quick Recap

- **try-catch** → Handle runtime errors safely.
- **throw** → Manually trigger an exception.
- **throws** → Declare exceptions a method may cause.
- **File Reading/Writing** → Use BufferedReader and BufferedWriter for efficient text file operations.

Build a Student Management System in Java implementing collections to store and edit records:

Definition

A Student Management System is an application that allows you to add, update, delete, and display student records. By using collections like ArrayList, HashMap, or HashSet, we can store student data dynamically without worrying about fixed sizes like arrays.

- **Example Implementation:**

```
import java.util.*;
class Student {
    int id;
    String name;
    String course;
    Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }
    @Override
    public String toString() {
        return "ID: " + id + ", Name: " + name + ", Course: " + course;
    }
}
public class StudentManagementSystem {
    public static void main(String[] args) {
        // Using ArrayList to store student records
        List<Student> students = new ArrayList<>();
        // Adding students
        students.add(new Student(1, "Ravi", "Computer Science"));
```

```
        students.add(new Student(2, "Priya", "Mathematics"));
        students.add(new Student(3, "Arjun", "Physics"));
        // Display all students
        System.out.println("All Students:");
        for (Student s : students) {
            System.out.println(s);
        }
        // Editing a student record (update course of student with
ID=2)
        for (Student s : students) {
            if (s.id == 2) {
                s.course = "Statistics";
            }
        }


        // Deleting a student record (remove student with ID=3)
        students.removeIf(s -> s.id == 3);
        // Display updated list
        System.out.println("\nUpdated Students:");
        for (Student s : students) {
            System.out.println(s);
```

```
        }
    }
}
```

How It Works:

- ArrayList stores student objects dynamically.
- Add → students.add(new Student(...))
- Edit → Loop through list and update fields.
- Delete → Use removeIf() or iterator.
- Display → Iterate and print records.

☑ Quick Recap

- Collections make record management flexible and efficient.
- ArrayList is great for ordered records.
- HashMap could be used if you want fast lookup by student ID.
- HashSet ensures unique student entries.